

Object Oriented Programming – CS/CE 224/272

Nadia Nasir



Content

- How to build C++ program
 - Preprocessor directives
 - Header files
 - Using namespace std
 - User input
 - Loops
 - If/else
 - switch
 - Logical and arithmetic operators
 - Datatypes
 - int, size_t, double, float, bool, and char
 - Cast and type conversion
 - Definition and declaration
 - Setw
 - math functions
-

Preprocessor directives and Header files



- C++ treats input and output as a **stream** of characters.
 - **stream** : sequence of characters (printable or nonprintable)
 - The functions to allow standard I/O are in **iostream** header file.
 - **#include <iostream>**
 - instructs the preprocessor to include a section of standard C++ code, known as header iostream, that allows to perform standard input and output operations, such as writing the output of this program (Hello World) to the screen.
 - **namespace:**
 - a collection of names and their definitions. Allows different namespaces to use the same names without confusion.
-



main() function

The function named main is a special function in all C++ programs. It is the function called when the program is run.

The execution of all C++ programs begins with the main function, regardless of where the function is actually located within the code.

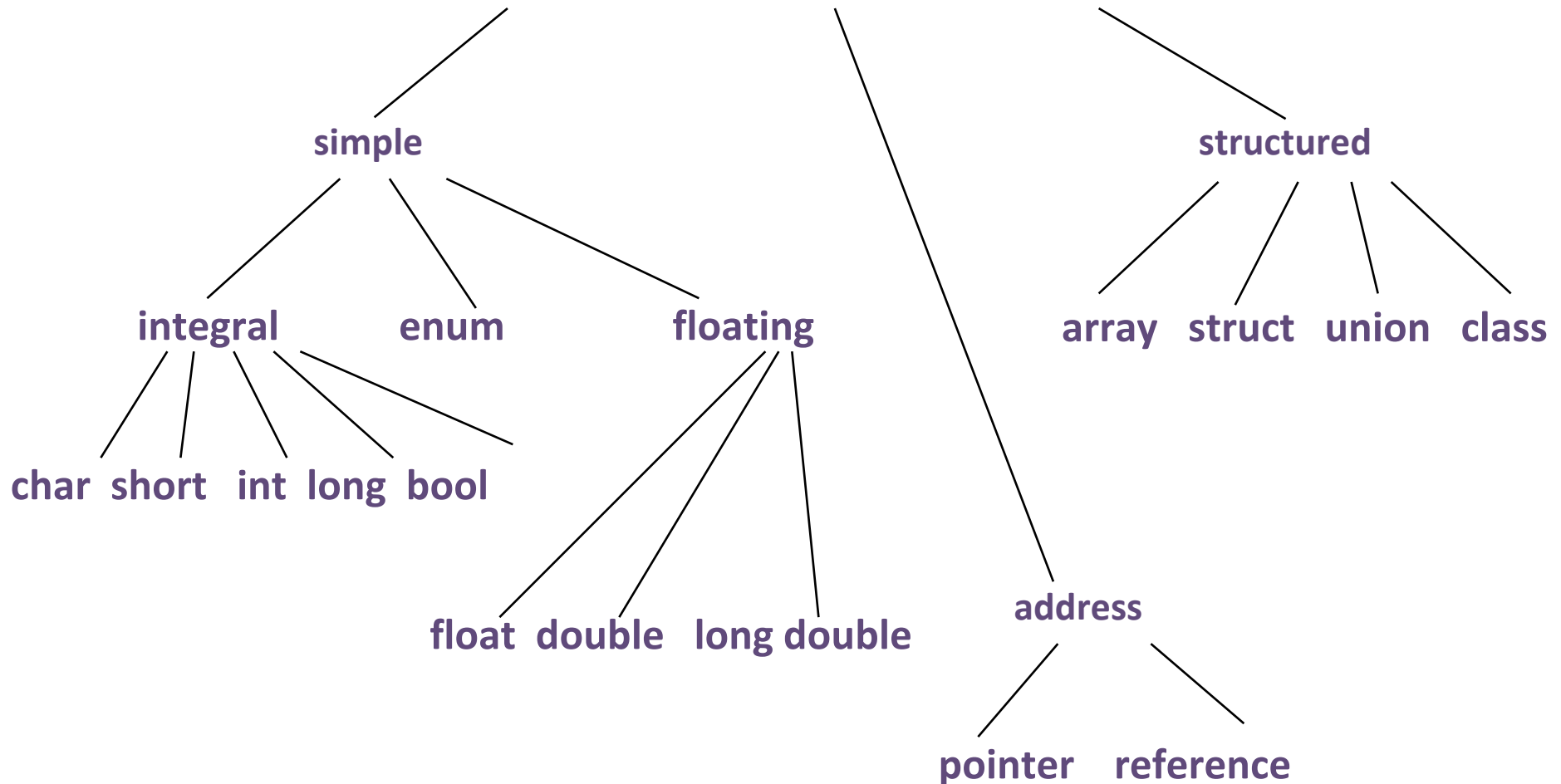


What is a Variable?

A variable is a memory address where data can be stored and changed.

Declaring a variable means specifying both its name and its data type.

C++ Data Types



Datatypes

- Signed integers types:
 - short, int, long, and long long
 - Unsigned integer types
 - unsigned short, unsigned int, unsigned long, unsigned long long
 - Decimal datatypes
 - float, double
 - Boolean data type
 - bool
 - Character data type
 - char
 - Nothing data type
 - void
-

Datatypes



```
int main(int argc, char* argv[])
{
    int i = 1000000;
    int invalidI = 1000000000000000;
    short s = 100;
    short invalidS = 100000;
    long l = 1000000000000000;
    std::cout << i << ", " << invalidI << '\n';
    std::cout << s << ", " << invalidS << '\n';
    std::cout << l << '\n';

    unsigned int ui0 = -1;
    unsigned int ui = 1;
    std::cout << ui0 << ", " << ui << '\n';

    float f = 1000.;
    float invalidF = 1000.123456789123456;
    std::cout << std::setprecision(15) << f << ", " << invalidF << '\n';

    double d = 1000.123456;
    double dd = 1000.12345678912;
    std::cout << d << ", " << dd << '\n';

    bool b = true; // false
    char a = 'A'; // char a = "A" --> error
    std::cout << a << '\n';

    int charAToInt = static_cast<int>(a);
    std::cout << charAToInt << '\n';

    return 0;
}
```


Type	Typical Bit Width	Typical Range
char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 126
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	Range	0 to 65,535
signed short int	Range	-32768 to 32767
long int	4bytes	-2,147,483,648 to 2,147,483,647
signed long int	4bytes	same as long int
unsigned long int	4bytes	0 to 4,294,967,295
float	4bytes	+/- 3.4e +/- 38 (~7 digits)
double	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	16bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	2 or 4 bytes	1 wide character

Cin-n-Cout

- Cin reads input from the stream

→ → → → → → → → → → →

```
int main(int argc, char* argv[])
{
    int n = 0;
    std::cin >> n;

    std::cout << "You typed:" << n << '\n';
}
```

- Cout is used to print to output stream

← ← ← ← ← ← ← ← ← ← ← ← ← ← ← ← ←

std::endl vs \n

- To print new line, you can use `std::endl`

```
int main(int argc, char* argv[])
{
    int n = 0;
    std::cin >> n;

    std::cout << "You typed:" << n << std::endl;
}
```

- OR you can use '\n'

```
int main(int argc, char* argv[])
{
    int n = 0;
    std::cin >> n;

    std::cout << "You typed:" << n << '\n';
}
```

- Use whichever approach you prefer. But using \n is better

Loopiness!

- Any loop can be used in any situation! It is just your preference.
- Some are preferred at times over other. You will know which ones to use as you practise!
- For loop. Conceptually similar to python. It runs a fixed amount of time

STARTING
POINT ←

```
int main(int argc, char* argv[])  
{  
    for (int i = 0; i < 5; ++i)  
    {  
        std::cout << i << '\n';  
    }  
  
    return 0;  
}
```

TEST

For Loop



2 semi colons

INCKEN

TEST

body of
for loop

```
int main(int argc, char* argv[])  
{  
    for (int i = 0; i < 5; ++i)  
    {  
        std::cout << i << '\n';  
    }  
    return 0;  
}
```

For loop

- Bare minimum for loop!
 - You only need 2 semicolons!
- Another variation
- There are many variations of for loop. You will learn them as you get more experienced with it

```
int main(int argc, char* argv[])
{
    for (;;)
    {
        std::cout << "hello world" << '\n';
    }

    return 0;
}
```

```
int main(int argc, char* argv[])
{
    int i = 0;
    for (; i < 5;)
    {
        std::cout << i << '\n';
        i++;
    }

    return 0;
}
```

While loop

- The structure of while loop is very similar to python

```
int main(int argc, char* argv[])
{
    int i = 0;
    while(i < 5)
    {
        std::cout << "i am i=" << i << '\n';
        ++i;
    }

    return 0;
}
```

Test condition
must be inside
parenthesis

- The output will be the same as the for loop we earlier saw

Do-while loop

- Consider that a homework and a reading exercise.
- Convert the earlier code into a do-while loop!

Decisions decisions!

- Familiar if/else to make decisions

```
int main(int argc, char* argv[])
{
    int i = 0;
    while( i < 10 )
    {
        if (i < 5)
        {
            std::cout << "hello\n";
        }
        else
        {
            std::cout << "world\n";
        }
        ++i;
    }

    return 0;
}
```

Body of if

Body of else

condition in parenthesis

Decisions decisions!

- You can also use if/else if/else
- What you learned in pfun about chained and nested conditionals holds here too
- An if statement can be nested inside another if or else if or else
- Like in python, you can't have an else without if

```
int main(int argc, char* argv[])
{
    int i = 0;
    for( int i = 0; i < 10; ++i )
    {
        if (i < 5)
        {
            std::cout << "hello\n";
        }
        else if (i < 10)
        {
            std::cout << "C++\n";
        }
        else
        {
            std::cout << "world\n";
        }
        ++i;
    }

    return 0;
}
```

Homework



- What this code will do?

```
int main(int argc, char* argv[])
{
    int i = 0;
    for( int i = 0; i < 10; ++i )
    {
        int num = -1;
        std::cout << "enter number: ";
        std::cin >> num;
        std::cout << "square of that number is" << num*num << '\n';
    }

    return 0;
}
```

- Take input from a user compute the square of a number. User should be prompted appropriately, and the output should be printed out. The program should exit when user just presses enter



Functions in C++

A function is a group of statements that together perform a task. Every C++ program has at least one function which is `main()` but we can define additional functions as well.

You can divide up your code into separate functions; how you divide up your code among different functions is up to you, but logically the division usually is such that each function performs a specific task.

A C++ function definition consists of a function header and a function body. Here are all the parts of a function:

- **Return Type** - A function may return a value. The `return_type` is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the `return_type` is “void”.
 - **Function Name** - This is the actual name of the function. The function name and the parameter list, together constitute the function signature.
 - **Parameters** - A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
-

- **Function Definition:** A function definition provides the actual body of the function. The function body contains a collection of statements that define what the function does.

```
return_type function_name ( parameter1 , parameter2 , parameter3 ,... )  
{  
    body of the function  
}
```

- **Declaring a function:** A function declaration tells the compiler about a function's name, return type, and parameters. The actual body of the function can be defined separately.

```
return_type function_name ( parameter list );
```

- **Calling a function:** To use a function, you will have to call or invoke that function. When a program calls a function, program control is transferred to the called function. A called function performs defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns program control back to the main function.

Example



```
#include <iostream>
using namespace std ;
// function declaration
int max (int num1 , int num2 );

int main ()
{
    // local variable declaration :
    int a = 100;
    int b = 200;
    int ret ;
    // calling a function to get max value .
    ret = max (a , b );
    cout << " Max value is : " << ret << endl ;
    return 0;
}

// function returning the max between two numbers
int max (int num1 , int num2 )
{
    // local variable declaration
    int result ;
    if ( num1 > num2 )
        result = num1 ;
    else
        result = num2 ;
    return result ;
}
```

Switch Statement

- The *switch statement* provides another way to decide which statement to execute next.
- The *switch* statement evaluates an expression, then attempts to match the result to one of several possible *cases*.
- Each case contains a value and a list of statements.
- The flow of control transfers to statement associated with the first case value that matches.

```
#include <iostream>
void printDigitName(int x)
{
    switch (x)
    {
        case 1:
            std::cout << "One";
            break;
        case 2:
            std::cout << "Two";
            break;
        case 3:
            std::cout << "Three";
            break;
        default:
            std::cout << "Unknown";
            break;
    }
}

int main()
{
    printDigitName(2);
    std::cout << '\n';
    return 0;
}
```


break statement

- Often a *break statement* is used as the last statement in each case's statement list.
 - A *break* statement causes control to transfer to the end of the *switch* statement.
 - If a *break* statement is not used, the flow of control will continue into the next case.
 - Sometimes this may be appropriate, but often we want to execute only the statements associated with one case.
-

default label in switch statement

- A `switch` statement can have an optional *default case*
 - The default case has no associated value and simply uses the reserved word `default`
 - If the default case is present, control will transfer to it if no other case value matches
 - If there is no default case, and no other value matches, control falls through to the statement after the switch
-

Switch vs if-else

Switch statements are best used when there is a single expression we want to evaluate for equality against a small number of values.

A switch-statement can be more readable, makes it clearer that it is the same expression being tested for equality in each case, and has the advantage of only evaluating the expression once (making it more efficient).

However, if-else is significantly more flexible. Some cases where if or if-else is typically better:

- Testing an expression for comparisons other than equality (e.g. `x > 5`)
 - Testing multiple conditions (e.g. `x == 5 && y == 6`)
 - Determining whether a value is within a range (e.g. `x >= 5 && x <= 10`)
 - The expression has a type that switch doesn't support (e.g. `d == 4.0`).
 - The expression evaluates to a bool.
-

Concepts covered so far

- `iostream` , namespace `std`
 - For, while, and do while loop
 - `cin`, `cout`, `endl`, `setw`, `setprecision`
 - Datatypes , size and its range
 - Signed, and unsigned int
 - Decimal types
 - Bool, char and void
 - If/else if/else
 - Switch statement
-